

RL-Based Memory Pooling for Octopus CXL Systems

8-minute advisor update

Pulak Mehrotra

March 1, 2026

1) What is RL?

- Reinforcement Learning (RL) learns a policy by trial-and-error interaction.
- At each decision event:

$$s_t \xrightarrow{\pi_\theta} a_t, \quad r_t, \quad s_{t+1}$$

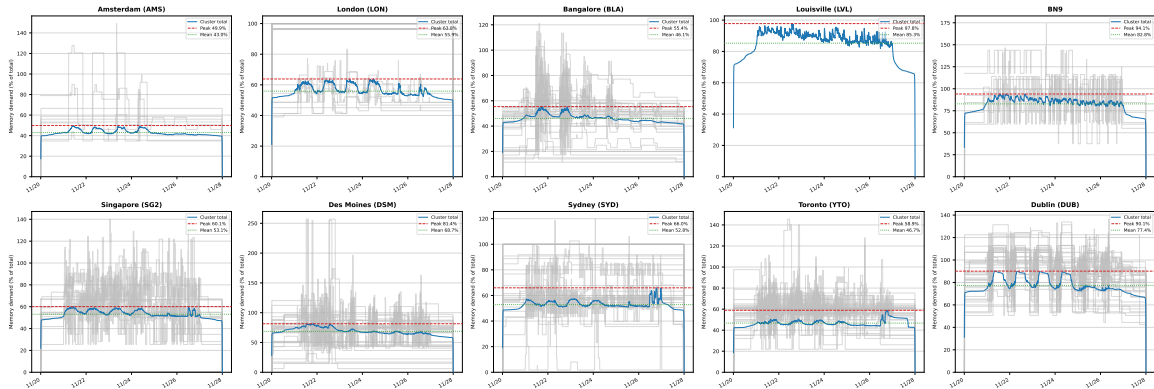
- Objective:

$$\max_{\pi_\theta} \mathbb{E} \left[\sum_{t=0}^T \gamma^t r_t \right]$$

- In this project, each VM arrival is one decision point.
- The policy learns how to split VM memory demand across reachable MPDs.

2) Why RL is a good fit (temporal trends)

Per-server (grey) and cluster-aggregate (blue) memory demand — 10 Azure datacenters
Red dashed = cluster peak; green dotted = cluster mean; y-axis = % of total physical DRAM



- Clear diurnal regularity means future demand is partially predictable.
- Time-aware allocation can reserve headroom before expected ramps.
- RL is well-suited to learning this anticipation from trace replay.

3) Policy Design: State

State table (cleaned-up design)

Signal	Current implementation	Next iteration
Per-accessible-MPD load	Normalized current loads	Add short-horizon deltas/trends
Topology connectivity	Binary reachable-MPD mask	Keep (hard validity constraint)
Request context	Normalized VM memory request	Keep and add request-type bucket
Global pressure	Current peak normalized MPD load	Keep and add rolling peak estimate
Temporal context	Hour-of-day sin / cos	Keep and add day-of-week encoding

Difference vs current implementation in `train.py/env`

- Current code is mostly instantaneous.
- Next iteration should add short history/trend features (e.g., load deltas) to improve anticipation.

4) Policy Design: Action

- Output is a continuous vector over reachable MPDs:

$$\mathbf{w} \in \mathbb{R}^k, \quad w_i \geq 0, \quad \sum_i w_i = 1$$

- Connectivity mask enforces invalid links $\rightarrow$ zero allocation.
- Physical allocation:

$$\text{alloc}_i = w_i \cdot \text{VM_memory}$$

- This keeps control high-level and low-overhead: no migration action in-loop.

Difference vs current code

- Current policy outputs unconstrained scores then applies softmax.
- We keep this but emphasize explicit masked simplex interpretation.

5) Policy Design: Reward

Primary objective: reduce required per-MPD peak capacity

$$\text{Savings} = 1 - \frac{m \cdot \max_j(\text{PeakLoad}_j)}{\sum_i \text{DRAM}_i}$$

Training reward (clean form)

$$r_t = -\Delta \max_j(c_j) - \lambda \cdot \text{Var}(\mathbf{c}_t)$$

where $\mathbf{c}_t$ is MPD load vector.

- First term penalizes increasing the worst MPD.
- Second term gives dense signal for load balancing.

Difference vs current `train.py` environment

- Current reward is close to peak-change only.
- Planned update adds variance shaping to reduce mode collapse.

6) Model Choice: Why SAC (and where PPO fits)

- **Chosen primary model: SAC**

- Continuous action support fits allocation vector directly.
- Off-policy replay improves sample efficiency on long traces.
- Entropy term helps exploration and avoids brittle deterministic policy.

- **PPO as secondary baseline**

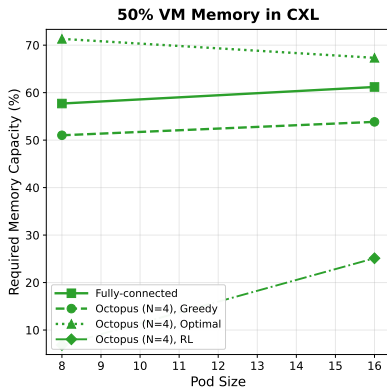
- On-policy, often more stable but less sample efficient here.

- **Practical note**

- Current `train.py` supports both SAC/PPO; experiments currently centered on SAC.

7) Results: Where RL stands now

Fully-Connected vs. Octopus (Greedy) vs. Octopus (RL) LON23PrdApp01



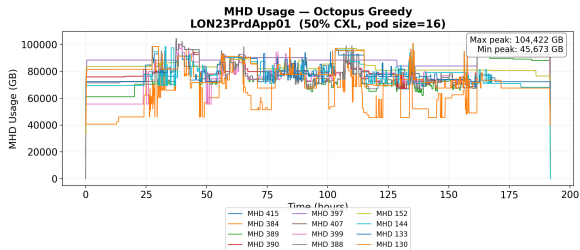
Current status

- RL is improving but not consistently closing greedy→optimal gap.
- Best gains appear in temporally structured regimes.

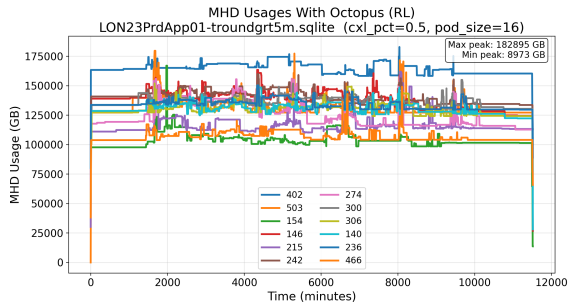
Message for this update

- We are getting there with RL.
- Next step: reward/state refinement + longer training for robust gains.

8) Results: Load balancing behavior (Greedy vs RL)



Greedy



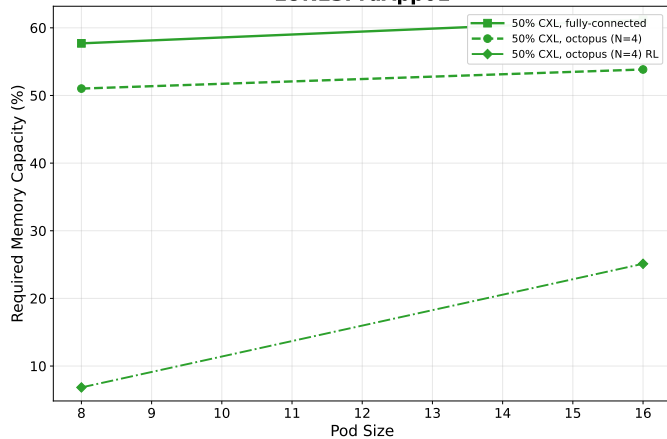
RL (SAC)

Takeaway

RL can still over-concentrate load on a few MPDs; improving reward shaping and temporal features is key to closing the remaining performance gap.

9) Results: Topology-level trend

LON23PrdApp01



Reading this plot

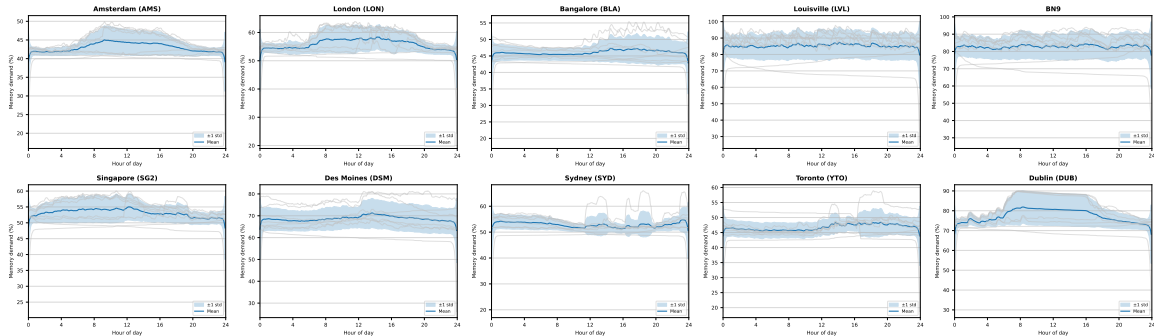
- Fully-connected remains an upper benchmark.
- Sparse Octopus is harder due to constrained routing.
- Gap widens at higher CXL fraction.

Takeaway

- There is measurable headroom to recover via better policies.

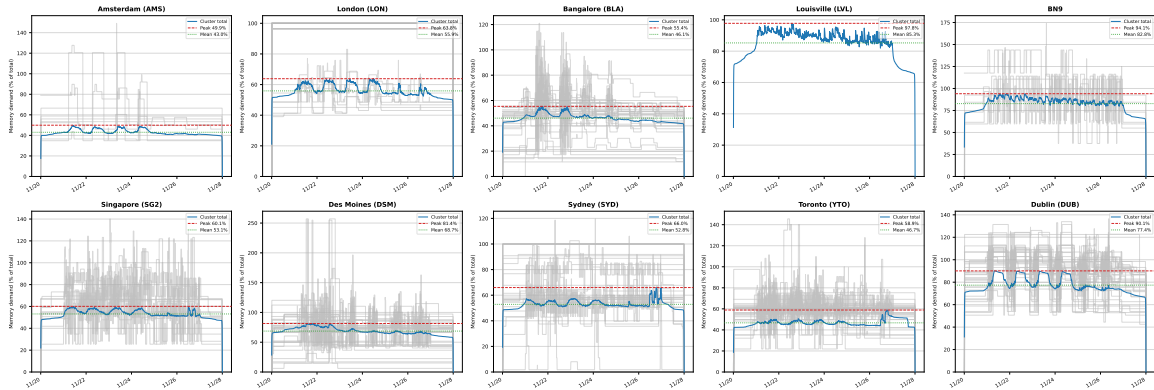
Appendix A: Diurnal trends (backup)

Diurnal memory demand patterns — mean (blue) $\pm$ 1 std, individual days in grey
y-axis = % of total physical DRAM; x-axis = hour of day (UTC)



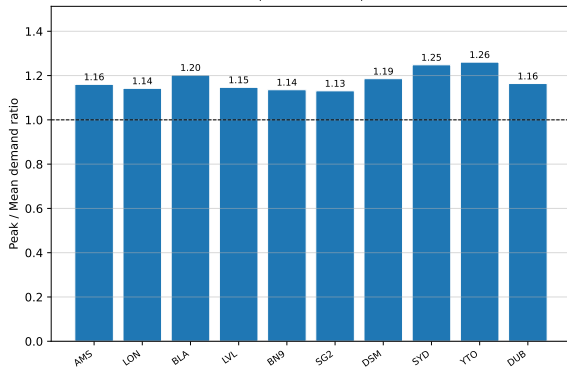
Appendix B: Additional temporal evidence

Per-server (grey) and cluster-aggregate (blue) memory demand — 10 Azure datacenters
Red dashed = cluster peak; green dotted = cluster mean; y-axis = % of total physical DRAM

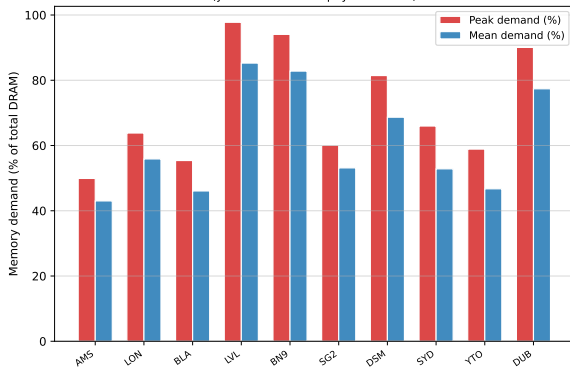


Appendix C: Peak-to-mean context

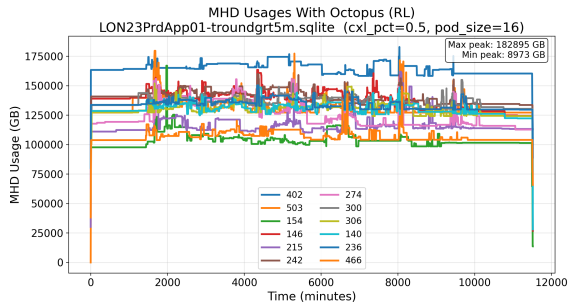
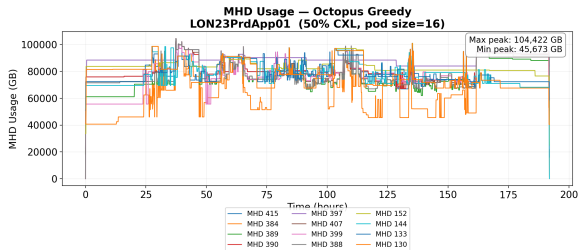
Peak-to-mean ratio per datacenter
(red = ratio > 1.4)



Peak vs. mean cluster memory demand
(y-axis = % of total physical DRAM)



Appendix D: Load-balancing behavior details



Useful for discussing where RL still diverges from balanced allocation.